

Maintenance & Developer Guide: Wendy's Spider Solitaire

****Current Version:**** v1.1.0

****Architecture:**** Single-Page Application (SPA)

****Tech Stack:**** HTML5, Vanilla JavaScript (ES6+), CSS3 (Tailwind CSS via CDN), Canvas Confetti (CDN)

1. Project Overview

Wendy's Spider Solitaire is a client-side, zero-build-step web application. It relies entirely on standard browser APIs (DOM, Pointer Events, LocalStorage). There is no backend server required. The game is driven by a central `state` object, and the DOM is completely redrawn upon every interaction.

2. Version History & Changelog

v1.1.0 (Current) - "The Polish Update"

****Infinite Celebration Loop:**** Integrated `canvas-confetti` to run on an infinite loop upon winning.

****Maintenance Note:**** Implemented `clearInterval()` cleanup functions inside `initGame()` and `replayBtn` listeners to prevent critical memory leaks when restarting the game.

****Magnetic Drop Zones:**** Added fallback logic to `elementFromPoint()`. If a user drops a card outside the physical `tableau-pile` boundary, the engine calculates the X-coordinate to snap the card into the intended column.

****Delta-Save Architecture:**** Replaced deep-cloning of the entire board with "Delta" snapshots. Only columns modified during a move are saved to the history stack. This reduced `localStorage` footprint by ~80% and allowed for infinite Undo/Redo capabilities without crashing the browser.

****Mobile Touch Overrides:**** Added `overscroll-behavior: none` to the `html` and `body` tags to strict-block Android pull-to-refresh and iOS swipe-to-go-back gestures.

****Permanent Win Banner:**** Upgraded the `showMessage()` utility to accept a `permanent` boolean, preventing the "Great Job Wendy!" banner from auto-hiding.

v1.0.0 - "Initial Release"

* Core game engine (1, 2, and 4 suit logic).

* Drag-and-drop pointer physics.

* Click-to-move accessibility fallback.

* Removed the strict traditional rule preventing dealing on empty columns to allow for casual play.

3. Maintenance Guidelines & Best Practices

A. Updating the DOM (The UI Pipeline)

The golden rule of this application: ****Never manually manipulate the DOM for game logic.****
If you want to move a card, change a score, or update a stock pile, you must:

1. Update the ``state`` object variables.
2. Call ``updateUI()``.

The ``updateUI()`` function completely obliterates the innerHTML of the game board and rebuilds it based on the newest state.

****Warning:**** Never call ``updateUI()`` while the ``drag.active`` boolean is true. Redrawing the DOM while a user is holding a card will instantly delete the DOM node from under their finger, causing the pointer engine to crash.

B. Dependency Management (CDNs)

The application relies on two external libraries loaded via Content Delivery Networks (CDNs):

1. ****Tailwind CSS:**** `<script src="https://cdn.tailwindcss.com"></script>`
2. ****Canvas Confetti:**** `<script`

`src="https://cdn.jsdelivr.net/npm/canvas-confetti@1.9.2/dist/confetti.browser.min.js"></script>`

****Maintenance Note:**** If the game is played on a device without internet access, the styling and fireworks will fail to load. For a true offline-capable PWA (Progressive Web App), these scripts would need to be downloaded and hosted locally.

C. Troubleshooting the Pointer Engine

We use Pointer Events (``pointerdown``, ``pointermove``, ``pointerup``, ``pointercancel``) instead of standard Mouse or Touch events because standard MDN compliant browsers unify them.

***** If cards get "stuck" to the screen on mobile, it is usually because the OS fired a ``pointercancel`` event (e.g., a system notification popped up during a swipe). Ensure the ``pointercancel`` event listener is always clearing the ``drag`` object and re-running ``updateUI()``.

D. CSS and Responsive Math

****Container Queries (`cqi`):**** The numbers and suit icons inside the cards use ``clamp()`` and ``cqi`` (Container Query Inline) units. This is a modern CSS standard that scales the text based on the width of the *card itself*, rather than the viewport. If you change the aspect ratio of the cards, you must adjust the ``clamp()`` math to prevent text overflow.

4. Future Roadmap Ideas (Community Vetted)

****PWA Manifest:**** Add a ``manifest.json`` and a Service Worker to allow users to "Install" the game to their phone's home screen.

****Animation Tweening:**** Use the FLIP (First, Last, Invert, Play) animation technique to make cards smoothly slide to their new positions instead of teleporting instantly after a drop.

****Save Exporter:**** Add a button to ``JSON.stringify(localStorage)`` and trigger a ``txt`` file download so users can back up their perfect games.